



PROJECT-BASED LEARNING

PHASE-I EVALUATION

RESQ

Intelligent Disaster Rescue & Emergency Response Simulator

"Algorithms That Save Lives"

Team ID: DSCPP-III-2026-T117 | Mentor: Mr. Prabhdeep Singh

Department of Computer Science & Engineering

Graphic Era (Deemed to be University), Dehradun

Academic Session 2026-27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T117
Semester: 3rd
Sections: ML-1, ML-2, ML-4, DS-2
Domain: Emergency Response Systems (DSA + OOP)

Team Members

1.	Vanya Rathi (Team Leader)	2510383410
2.	Akshat Dobhal	2510370353
3.	Ayush Kandwal	2510380235
4.	Nitin Singh	2510370229

Mentor

Mr. Prabhdeep Singh

Interactions completed: 02

Problem Statement & Motivation

Problem Statement

In an earthquake, flood, fire or big road accident, a control room gets far more calls than the teams, ambulances and beds it actually has free. The order of dispatch is decided by hand, mostly in the order the calls came in, and the map app keeps sending units down roads that have already collapsed. So a critical victim waits behind a stable one and the golden hour goes by. RESQ adds the layer that is missing here: **who to rescue first, which unit to send, and which road is still open.**

Motivation

- ▶ An operator cannot re-rank hundreds of live calls by hand.
- ▶ Map and CAD tools assume the roads are still intact.
- ▶ A cut-off sector is usually found only after a unit gets stuck.

Target Users / Stakeholders

- ▶ Control rooms (112 / 108 dispatchers).
- ▶ NDRF / SDRF teams, ambulance crews, hospitals.
- ▶ Students who want to see DSA used on a real problem.

Objectives & Proposed Solution

Project Objectives

1. Take in rescue calls and keep every victim record in a linked list.
2. Sort victims with a max-heap priority queue, using severity plus how long they have waited.
3. Keep the disaster area as a weighted graph and find the shortest route with Dijkstra.
4. Use BFS to spot isolated sectors, so cut-off victims can be sent to air support.
5. Put the whole simulation inside a C++ OOP engine and keep a stack as the audit trail.

Key Features

- ▶ Four scenarios: earthquake, flood, fire, highway pile-up.
- ▶ Disaster-aware edge weights (epicentre slowdown, collapsed roads).
- ▶ RED victims routed only to trauma-capable hospitals.
- ▶ "CUT OFF (BFS)" flag for isolated sectors.

Proposed Approach

1. **Input:** rescue call – name, age, location, severity (RED/YELLOW/GREEN).
2. **Triage:** max-heap priority queue; FIFO queue for routine calls.
3. **Map:** weighted adjacency graph of the disaster area; blocked roads removed.
4. **Routing:** Dijkstra for unit → victim → hospital; BFS for reachability.
5. **Output:** route plan, ETA and a live operations dashboard.

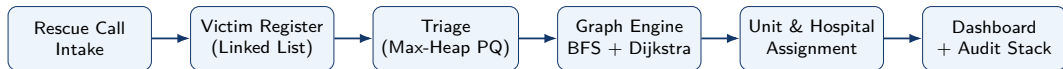
Key Objective: On the same scenario, reach RED (critical) victims faster on average than a plain first-come-first-served dispatch does.

Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures in C	Linked list (victim records), queue (routine calls), max-heap priority queue (triage), stack (operation history)	linked_list.c, queue.c, priority_queue.c, stack.c
Data Structures in C	Graph (disaster map), BFS (area exploration), Dijkstra (shortest route), insertion & quick sort, binary & linear search	graph.c, sort_search.c
OOPs with C++	Classes & objects (<i>Victim</i> , <i>Location</i> , <i>Hospital</i> , <i>Disaster</i>), encapsulation, constructors/destructors	resq.hpp
OOPs with C++	Inheritance & polymorphism — abstract <i>RescueUnit</i> specialised into <i>RescueTeam</i> / <i>Ambulance</i> ; <i>RescueOperation</i> engine	resq.cpp, main.cpp

Integration: the C layer gives us the data structures and algorithms, and the C++ layer puts them together into a single object-oriented simulation engine.

System Architecture / Workflow



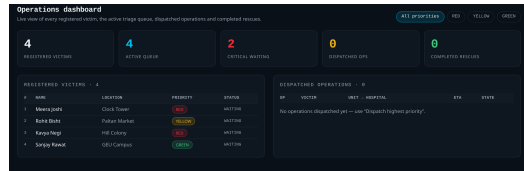
Major Components

- ▶ C DSA core — list, queue, heap, stack, graph.
- ▶ C++ OOP engine — units, hospitals, disasters, operations.
- ▶ Visual simulator — map, queue view, dashboard.

Data / Control Flow

- ▶ A call is saved, given a score and pushed into the heap.
- ▶ The disaster changes the edge weights, or blocks edges completely.
- ▶ Dijkstra returns the route, and the finished operation goes on the history stack.

Live Demo — Route Map & Operations Dashboard



Dashboard: registered victims, active queue, critical victims waiting, dispatched operations and completed rescues, filterable by priority.

Max-heap re-orders the queue by severity; Dijkstra routes the unit to the victim and then to the hospital, around collapsed roads.

Technology Stack & Methodology

Technology Stack

- ▶ Programming: C (C11) for DSA, C++ (C++17) for OOP
- ▶ Build: GNU Make, gcc/g++ with `-Wall -Wextra`
- ▶ Visual demo: browser-based simulator mirroring the same algorithms
- ▶ IDE: VS Code
- ▶ Version Control: Git / GitHub

Development Methodology

1. Requirement analysis of dispatch workflow
2. System design — data structures & class hierarchy
3. Incremental implementation, module by module
4. Testing on three disaster scenarios
5. Evaluation against FCFS baseline

We picked C because it gives direct control over memory and pointer-based structures, and C++ because inheritance and polymorphism let us add a new type of rescue unit without touching the algorithms.

Initial Progress & Team Contribution

Progress Achieved

- ▶ Read up on triage protocols and how dispatch works
- ▶ Fixed the requirements and the test scenarios
- ▶ Drew the architecture and the class diagram
- ▶ Wrote the C modules: list, queue, heap, stack and graph
- ▶ Got the C++ engine and the visual demo running end to end

Individual Contribution

Member	Role	Contribution
Vanya Rathi	Team Lead	Architecture, triage logic, integration, documentation
Akshat Dobhal	Developer	Linked list, queue, priority queue, stack
Ayush Kandwal	Developer	Graph module, BFS & Dijkstra routing
Nitin Singh	Dev / Testing	C++ classes, scenario testing, dashboard & reports

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: proposal, design, core DSA modules
2. Phase-II: C++ engine, multi-unit allocation
3. Phase-III: final simulator, comparison, demo

Expected Outcomes

- ▶ A working C/C++ simulator with a visual demo
- ▶ Measured time saved for critical victims over FCFS
- ▶ A log of every rescue operation that was carried out

References

1. E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, 1959.
2. T. H. Cormen et al., *Introduction to Algorithms*, 4th ed., MIT Press.
3. B. Stroustrup, *The C++ Programming Language*, 4th ed., Addison-Wesley.
4. National Disaster Management Authority (NDMA), India — guidelines on medical preparedness and mass-casualty triage.

Thank You
Questions & Suggestions